

Validating LLM-Based Localisation for Mining SQL Antipatterns in jOOQ Repositories

Kristo Isberg · Erki Eessaar

Abstract SQL antipatterns are recurring database design and query mistakes that can degrade maintainability, correctness, and performance. Detecting them in application code is difficult when domain-specific libraries such as jOOQ construct SQL statements dynamically. We study line-level localisation of SQL antipatterns in jOOQ-based Java code and use an LLM detector to investigate their occurrence in open-source projects. We construct a manually annotated dataset of 1,562 occurrences from 61 GitHub projects and evaluate multi-label, multi-occurrence localisation using line-span Intersection over Union, precision, recall, and F1-score. On the project-disjoint test set, the selected detector achieves a weighted F1-score of 0.88 against the original human ground truth. Applied to 602 projects containing 17,450 relevant files, it flags 15,931 occurrences of seven SQL antipatterns. Implicit Columns and ID Required appear in 535 and 523 projects, respectively. Exploratory analysis describes raw co-detection patterns across projects and files. API-level analysis associates many query-antipattern occurrences with a small set of jOOQ convenience methods. These detector-output measurements for jOOQ projects support a hybrid design in which deterministic analysis handles stable source patterns and LLM localisation handles cases requiring source context.

Keywords Empirical software engineering · SQL antipatterns · Static analysis · Large language models · Mining software repositories · jOOQ

K. Isberg
Tallinn University of Technology, Tallinn, Estonia
E-mail: krisbe@taltech.ee

E. Eessaar
Tallinn University of Technology, Tallinn, Estonia
E-mail: erki.eessaar@taltech.ee

1 Introduction

SQL remains one of the most widely used languages in software systems, yet recurring database design and query mistakes continue to appear in production code. These recurring mistakes are commonly described as SQL antipatterns: familiar but flawed solutions to database problems (Karwin 2023). Prior work has reported SQL antipatterns in both open-source and industrial systems and has linked them to maintainability and quality concerns (Muse et al. 2020; Sharma et al. 2018; Lyu et al. 2019; Dintyala et al. 2020). Despite this evidence, tool support remains uneven, especially when database access code does not contain plain SQL strings.

This paper focuses on jOOQ, a Java library for constructing SQL statements through a type-safe domain-specific language. jOOQ improves integration between Java and relational databases, but it also complicates static analysis: SQL statements can be built through fluent APIs, generated schema classes, intermediate Java variables, and conditional control flow. Traditional approaches that first extract SQL statements and then parse them can therefore miss behaviours that are visible in the source code but hard to reconstruct as complete SQL statements (Nagy and Cleve 2017; Meurice et al. 2016). Large language models (LLMs) offer a possible alternative because they can reason over source-level context without requiring a complete language-specific extractor for every dynamic query pattern.

We evaluate SQL-antipattern detection as a localisation task in which a detector identifies both the antipattern type and its line span. The measurement therefore corresponds to the warning sites later counted in repositories. Developers can inspect a concrete projection, predicate, or generated-schema declaration. Tool builders can route stable API patterns to deterministic rules and reserve semantic cases for contextual analysis.

Our contribution is a linked measurement design that validates the detector at the same granularity used for repository counting. SQLInspect-style approaches extract SQL from host-language code before applying specialised rules, while API- or abstract-syntax-tree-based rules encode stable source patterns directly (Nagy and Cleve 2017, 2018; Muse et al. 2020). Recent LLM code-smell studies commonly evaluate recognition or classification, and other work tests LLMs on conventional static-analysis tasks (Sousa et al. 2025; Sadik and Govind 2025; Alomari et al. 2026; Su and McMillan 2026).

We evaluate multi-label, multi-occurrence line-span localisation in dynamic jOOQ code on a project-disjoint held-out set. We then carry the measured class-specific limitations into repository-level prevalence, exploratory co-detection, and API-association findings. The resulting claims concern the evaluated localisation task and detector outputs; deterministic rules and independent population validation remain necessary for their respective purposes.

The study has two connected goals. First, we evaluate whether LLMs can localise SQL antipatterns in jOOQ-based Java code with sufficient accuracy to serve as a research instrument. Second, we use the resulting detector to

analyse the prevalence, co-occurrence, and API-level manifestations of SQL antipatterns across a large set of open-source jOOQ projects.

We address the following research questions:

- RQ1: How accurately can LLMs localise SQL antipattern occurrences in jOOQ-based Java code?
- RQ2: How prevalent are SQL antipatterns in open-source jOOQ projects?
- RQ3: Which SQL antipatterns co-occur across projects and files?
- RQ4: Which jOOQ API methods are most frequently associated with query antipattern occurrences?

We separate RQ3 from RQ4 because they capture different empirical dimensions of the same phenomenon. RQ3 studies project- and file-level relationships among antipatterns, whereas RQ4 studies concrete source-level API patterns associated with query antipattern occurrences.

The paper makes the following contributions:

- a manually annotated dataset of 1,562 SQL-antipattern occurrences in 61 jOOQ projects;
- an evaluation of LLM-based SQL-antipattern detection as a multi-label, multi-occurrence localisation task;
- detector-output measurements for seven SQL antipatterns across 602 open-source jOOQ projects;
- exploratory co-detection and jOOQ API-association evidence that can inform documentation, developer guidance, and future static-analysis tools.

2 Background and Related Work

2.1 SQL Antipattern Detection

SQL antipatterns describe recurring database design, query, and application-development mistakes (Karwin 2023). Prior empirical work has shown that such antipatterns are present in real systems and can persist for long periods (Muse et al. 2020). Sharma et al. studied SQL smells in database-centric applications, while Muse et al. analysed the prevalence of SQL antipatterns using SQLInspect, a static-analysis tool that extracts SQL statements from Java code before detecting selected antipatterns (Sharma et al. 2018; Muse et al. 2020; Nagy and Cleve 2017, 2018).

SQL antipatterns vary in the evidence needed for detection. Some are visible in a single query, such as selecting every column when only a subset is required. Others concern schema design, including generic surrogate-key naming, missing foreign keys, inappropriate use of floating-point types, encoded value domains, or ambiguous use of nullability. Still others depend on developer intent: a `LIKE` predicate may implement legitimate prefix search or may be an inefficient substitute for full-text search. Consequently, a detector must do more than find keywords. It must define which source representation is in

scope, translate database-level concepts into observable source-level rules, and distinguish syntactic signals from decisions requiring domain context.

Prior work also differs in the unit being measured. Schema-oriented techniques can inspect catalogues or DDL and reason about constraints independently of application execution. Query-smell studies may count affected statements, files, occurrences, or projects, each answering a different question. Project presence captures dissemination but treats one and hundreds of occurrences alike; occurrence counts capture intensity but correlate with project size; statement-normalised rates depend on how statements are extracted or approximated. Prevalence values therefore cannot be compared safely without examining their denominator and source representation.

An antipattern denotes a recurring solution with recognised liabilities. Local workload, compatibility, or domain constraints may nevertheless justify the trade-off, and static tools rarely possess enough execution evidence to assess harm in every instance. We therefore annotate conformance to explicit operational definitions. Detector precision measures agreement with that codebook; it does not estimate the proportion of warnings that developers must change.

Query extraction fits embedded SQL strings but depends on extractor completeness. Meurice et al. report query-extraction precision of at least 0.879 and recall of at least 0.715 across evaluated systems (Meurice et al. 2016); Nagy et al. and Muse et al. consequently treat extraction quality as a limiting factor for downstream antipattern detection (Nagy and Cleve 2017; Muse et al. 2020). This limitation is especially relevant for jOOQ, where generated schema classes, fluent API calls, and host-language control flow can express database behaviour without directly parseable SQL strings.

Extraction and source-level analysis address complementary cases. Reconstructing a query enables mature SQL parsers and deterministic rules, produces repeatable warnings, and may expose relationships obscured by a fluent call chain. Direct source analysis can incorporate Java-level context, generated-schema metadata, and library semantics without complete query reconstruction. An LLM detector, however, has weaker determinism, higher runtime and monetary cost, and less transparent failure modes. Comparisons must therefore account for differences in representation, supported antipatterns, and localisation units. We use extraction-based work to establish the problem and comparison boundaries instead of treating it as an equivalent baseline.

Rule-based and learned approaches incur different maintenance costs. A parser and rule set must be updated when a host library introduces a construction pattern, but its accepted language and warning condition remain inspectable. A general LLM can recognise unenumerated forms without a dedicated parser, yet behaviour may change with model snapshots or provider infrastructure. That flexibility is useful for empirical mining only when validation is reported at the same granularity as later measurements. We consequently treat the LLM as a versioned measurement instrument and retain deterministic post-processing for schema validation, line matching, and aggregation.

Recent LLM-based smell and vulnerability detectors provide a second point of comparison. De et al. evaluate LLMs for PL/SQL smell detection, including a smell corresponding to the `SELECT *` subset of Implicit Columns (Sousa et al. 2025). Other studies evaluate LLMs for broader code-smell or vulnerability detection tasks (Sadik and Govind 2025; Tamberg and Bahsi 2024; Blaauwendraad 2025). These studies show that LLMs can be useful for code-quality detection, but most reported tasks are classification-oriented. Our study differs by evaluating SQL-antipattern detection as a line-span localisation task in Java source code using jOOQ.

The relevant literature supplies complementary baselines. SQL-smell mining motivates the target phenomena and prevalence questions; extraction research identifies a coverage bottleneck; code-smell LLM work motivates semantic source analysis; and localisation research motivates actionable output. Their labels and evaluation units differ, so their reported metrics cannot jointly validate occurrence counts for dynamic jOOQ code. We use each body of work for the part of the design it supports.

2.2 jOOQ and Dynamic Database Access Code

jOOQ represents SQL concepts as Java API calls and generated schema classes. This design helps developers construct type-safe queries, yet antipatterns can arise through combinations of generated code, fluent method calls, and raw SQL fragments. For example, blind projections related to the Implicit Columns antipattern can arise from both convenience methods that fetch all fields from a table and explicit `SELECT *` strings. The jOOQ documentation discourages blind projections when redundant data should be avoided (joo 2025), yet such calls are among the most convenient ways to retrieve generated records.

Generated schema classes expose antipatterns such as generic primary keys or missing constraints. Other antipatterns appear in DML/DQL query construction code. A jOOQ detector therefore needs to inspect both generated classes and hand-written database access code under the distinct antipattern definitions applicable to each file category.

This split has methodological consequences. Generated table classes provide a regular representation of columns, data types, defaults, and keys, but the representation reflects what the code generator could observe and what a project committed to version control. Hand-written query files are more heterogeneous: fluent calls can span methods, use intermediate variables, mix generated fields with raw SQL, and place selection and execution far apart. A single generic prompt would either expose irrelevant schema antipatterns to query files or ask the model to infer unavailable query behaviour from generated classes. We therefore treat file type as part of the detector’s operational context and evaluate the combined output at the occurrence level.

API-specific rules remain a plausible alternative for several frequent cases. Calls such as `selectFrom(TABLE)` or `select().from(TABLE)` can often be recognised without an LLM, and a production tool should prefer determin-

istic checks where their semantics are sufficient. Harder cases motivate the broader detector: aliases and intermediate expressions obscure the final projection; nullable defaults require interpreting generated column declarations; potential foreign-key omissions require relating a column to available keys; and wildcard predicates require distinguishing search intent. Our evaluation measures whether one source-level instrument can cover this heterogeneous family. Individual occurrences may still admit deterministic detection.

2.3 LLMs for Code Analysis and Localisation

Recent work has applied machine learning and LLMs to code translation, code smell detection, defect prediction, and other software engineering tasks (Fontana et al. 2016; Sousa et al. 2025; Sadik and Govind 2025). For the present study, the key methodological distinction is localisation. A detector that returns only a file-level label is less actionable than one that identifies the affected line span. We therefore evaluate LLM outputs using line-span overlap and classification metrics, aligning the task more closely with localised defect prediction and warning generation than with binary file classification.

LLM code-smell work establishes the value of source-level semantic cues and narrows the available novelty claim. De et al. evaluate LLMs for PL/SQL smells, including a smell related to the `SELECT *` subset of Implicit Columns (Sousa et al. 2025). Alomari et al. compare in-context learning with fine-tuning for code-smell detection and study performance, generalisation, data, and computational trade-offs (Alomari et al. 2026). Su and McMillan test code LLMs on call-graph, AST, and data-flow tasks. They find that competence on programming tasks does not imply conventional static-analysis ability (Su and McMillan 2026). Generic code understanding and code-smell classification are therefore established topics rather than the research gap addressed here.

Localisation changes both the output and the evaluation problem. A file can contain multiple occurrences of one antipattern and several antipattern types. Predictions may overlap the correct source region without matching its exact boundaries, and several predictions may compete for one reference occurrence. File-level accuracy and binary confusion matrices cannot represent these cases.

Line-level defect prediction offers a methodological analogy because it evaluates actionable source regions. Defect-proneness localisation and explicit antipattern-occurrence localisation, however, have different labels and evidence requirements. We therefore use span overlap and one-to-one assignment to operationalise warning localisation for this study.

The remaining gap is a linked empirical question: whether a source-context LLM can be validated transparently enough to support repository mining where deterministic extraction is incomplete, and what detector-output evidence that instrument produces about jOOQ projects. Detector performance alone would yield another model comparison. Prevalence without instrument

Table 1 Comparison of representative SQL/code-smell detection approaches and this study. “Repository use” distinguishes evaluation of a detector from using its occurrence outputs as measurements in a repository study.

Approach	Input	Labels/target	Localisation	Validation design	Repository use
SQLInspect	SQL extracted from host code	Selected antipattern rules	Extracted query or statement	Extraction and rule evaluation	Mines affected statements/projects
API/AST rules	Parsed source and API patterns	Rule-defined violations	Matched node or call	Deterministic rule evaluation	Scans source repositories
Recent LLM smell studies	Source snippets or PL/SQL	Smell classes	Commonly snippet or file label	Held-out model comparison	Primarily detector evaluation
This study	Java plus generated jOOQ schema source	Seven SQL antipatterns	Multi-label occurrence spans	Project-disjoint test	Detector-output mining of 602 projects

validation would leave the large-scale counts difficult to trust. The study design connects the two and keeps their uncertainties visible.

We test observable warning performance under a fixed source representation. The study leaves LLM internal representations, comparisons between fine-tuning and prompting, and developer usefulness outside its evidence. Its contribution is the empirical linkage of an operationalised heterogeneous task, occurrence-level validation, and a repository study that carries measured detector limitations into its conclusions.

Table 1 makes the comparison boundary explicit. It draws on SQLInspect and its repository study (Nagy and Cleve 2017, 2018; Muse et al. 2020), rule-based smell-detection methods (Marinescu 2004; Moha et al. 2010), and recent LLM smell studies (Sousa et al. 2025; Sadik and Govind 2025; Alomari et al. 2026). The rows describe different representations and validation targets rather than a controlled performance ranking. In particular, a deterministic rule test and a held-out LLM localisation evaluation do not estimate the same quantity.

3 Study Design

3.1 Repository Mining and Dataset Construction

We mined GitHub repositories for Java projects that use jOOQ and store generated jOOQ classes in version control. The generated classes were required because several database design antipatterns can only be detected from schema representations; compiling arbitrary GitHub projects to generate those classes would introduce substantial build and dependency failures.

A project qualified when it contained Java source using jOOQ for database access, committed generated classes representing its schema, and non-generated Java files referencing jOOQ or generated classes. Reconstructing generated classes for arbitrary repositories would have required project-specific Java versions, databases, private dependencies, and code-generator configurations. The

restriction makes both query and schema source available for analysis, but narrows the target population to projects that commit derived artefacts.

The mining process used the GitHub Code Search API (git 2022) to search Maven and Gradle manifest files for jOOQ dependencies. On 10 January 2026, the search identified 3,829 projects declaring a jOOQ dependency. Filtering for repositories with generated jOOQ classes reduced this set to 802 projects. Filtering again for non-generated Java files referencing jOOQ reduced the set to 645 projects. Manual, LLM-assisted duplicate and fork review removed 42 projects, resulting in 603 projects. A later duplicate removal yielded the final analysis set of 602 projects.

We searched Maven XML/POM and Gradle Groovy/Kotlin manifest syntax rather than Java imports. Import searches can be saturated by files from a few large repositories, while manifest matches identify dependency declarations more directly. Query variants covered the core artefact, code-generation and metadata artefacts, extensions, and common plugin syntax; the exact strings are included in the supplementary material. GitHub limits a code-search query to 1,000 results, so specific variants reduced overlap. Where a query still exceeded the limit, the retrieved set reflected GitHub’s most recently indexed results.

The duplicate review used repository file trees to generate possible clone groups with LLM assistance, followed by manual verification of every omission. The 42 removed repositories included tutorial-copy clusters and jOOQ derivatives. A duplicated project found during preparation of the full analysis was removed before prevalence statistics were calculated. These stages are reported separately so the change from 603 sampled candidates to 602 analysed projects is auditable.

This mining strategy is intentionally conservative. It favours projects where source-level analysis of both query code and generated schema code is possible. Its main limitation is incompleteness: the GitHub Code Search API limits each query to 1,000 results, and projects using other build tools or not committing generated jOOQ classes are outside the final population.

To construct the annotation sample, we defined a relevant database-access file as either a useful generated jOOQ class or a non-generated Java file referencing jOOQ/generated types. Known irrelevant generated paths, including records, POJOs, and external system schemas, were excluded. Head-tail breaks over relevant-file counts produced boundaries of 31 and 94 files: 467 projects were small, 100 medium, and 36 large. Using pseudo-random seed 123456, we sampled 10% from each group, selecting 47 small, ten medium, and four large projects.

A later corrected file filter changed the boundaries to 29 and 91 and the class sizes to 468, 98, and 36, but preserved the distribution shape. We retained the original 61-project sample to avoid selecting evaluation data retrospectively.

For project file count x and set S , the first head-tail boundary is the mean $\mu_1 = |S|^{-1} \sum_{x \in S} x$. The second is the mean μ_2 of projects above μ_1 . Projects at or below μ_1 are small, those between μ_1 and μ_2 medium, and those above



Fig. 1 Repository mining and filtering process. The final 602-project dataset contains open-source Java projects that declare a jOOQ dependency, store generated jOOQ classes in source control, and contain non-generated Java code referencing jOOQ.

μ_2 large. Sampling the same proportion within each class preserves rare large projects without making them dominate the manual workload. It does not make the sample representative of all possible jOOQ systems; it makes size coverage explicit within the mined frame.

This reproducible selection funnel defines the study corpus. It excludes projects using other build systems, projects hidden by the result cap, closed-source systems, and repositories generating schema classes only during builds. Accordingly, later prevalence estimates describe this corpus of open-source GitHub jOOQ projects.

3.2 Operational Definitions

The annotation covered 19 SQL antipatterns from Karwin’s catalogue (Karwin 2023) that were observable in static jOOQ source. We excluded Index Shotgun because selecting an index requires workload and execution-plan evidence; Spaghetti Query because no defensible source threshold distinguished necessary complexity from a smell; Pseudokey Neat-Freak and Diplomatic Immunity because they depend on organisational practices; and See No Evil because frameworks often handle database exceptions outside the analysed query file. Fear of the Unknown was limited to database-design evidence and query conditions whose null behaviour could be determined without tracing arbitrary Java inputs.

For each antipattern from the taxonomy (Karwin 2023), we recorded the project, file path, antipattern name, code fragment, line range, and reasoning. The final annotated dataset contained 1,562 occurrences. For model evaluation and large-scale analysis, we retained seven antipatterns with sufficient support across projects and occurrences: Implicit Columns, ID Required, Keyless Entry, Fear of the Unknown, 31 Flavors, Poor Man’s Search Engine, and Rounding Errors. We excluded antipatterns occurring in fewer than three projects or fewer than 25 total occurrences from later experiments to avoid conclusions from sparse classes.

Implicit Columns denotes a blind projection of every table field. It includes literal `SELECT *` and `jOOQ` calls that fetch all generated fields or complete records, but excludes count/existence operations whose projection is not returned as application data. *ID Required* denotes a synthetic primary key named only `id`, or an unnecessary surrogate where a stable, manageable natural key is represented; views are excluded. *Keyless Entry* denotes a likely reference column for which a suitable target key exists but no foreign key is represented, making it more contextual than a suffix match.

Fear of the Unknown includes special defaults used for missingness, nullable declarations for fields that always receive non-null values, and query predicates whose treatment of a nullable database field is demonstrably wrong; generic Java nullability is outside scope. *31 Flavors* covers enumerations or restrictive checks used instead of lookup tables, excluding emptiness and range checks. *Poor Man’s Search Engine* covers `LIKE`, case-insensitive `LIKE`, or regular expressions used for full-text-style search; legitimate prefix searches are excluded. *Rounding Errors* covers fixed-precision domain values represented by floating-point rather than decimal/numeric columns.

These rules adapt database-level descriptions to observable `jOOQ` constructs. For example, `selectFrom(TABLE)` expresses the blind projection of `SELECT *` without containing a wildcard string. Conversely, the presence of a method capable of expressing an antipattern is not sufficient: the code must meet the applicable contextual rule and exclusions. The full decision trees remain in supplementary material, while the main text records the boundaries needed to interpret the labels.

3.3 Annotation and Adjudication

One annotator with prior `jOOQ` and SQL-antipattern experience reviewed the 61 projects. Each occurrence was stored in a spreadsheet and exported to CSV with its type, repository, repository-relative file, inclusive start and end lines, code fragment, and rationale. The heterogeneous repositories were not executed because many required unavailable environments or dependencies and the target task was static analysis. Every sampled project contained an occurrence; the final dataset contained 1,562 occurrences of 19 antipatterns, including 795 Implicit Columns and 259 ID Required occurrences.

The annotation codebook was developed iteratively as decision trees. When review exposed a new variant or non-antipattern edge case, the corresponding tree was updated and earlier files potentially affected by the change were re-reviewed. This reduced reliance on memory or intuition, but it did not create independent expert consensus.

The ground truth was created by a single annotator. To assess internal consistency, a subset of 128 files was re-annotated after a 30-day washout period. The resulting Cohen’s kappa was 0.834, indicating high consistency over time. This value does not replace inter-annotator agreement; it is a consistency check rather than evidence of independent expert consensus.

The 128-file subset combined a seeded 10% sample of files containing occurrences with an equal number of randomly sampled files without occurrences. Re-annotations matched only when the label and inclusive line span were identical. A 16-by-16 confusion matrix represented 15 observed antipatterns plus background. The resulting $\kappa = 0.834$ therefore measures temporal consistency under exact localisation, not merely agreement that a file is positive.

No independent adjudicator was available. Later qualitative review of detector disagreements identified missed human occurrences and incorrect reference labels or spans, producing a corrected ground-truth variant. Because the detector selected which cases received extra scrutiny, the corrected variant is biased in its favour and cannot reveal cases missed by both sources. We retain the original labels as primary and use corrected results only as an optimistic sensitivity analysis.

3.4 Data Partitioning

The 61 annotated projects were assigned to training, validation, and test splits as whole projects rather than splitting files from the same project across different subsets. This avoids prompt development and evaluation sharing project-specific conventions. The split used a Monte Carlo seed search over one million candidate seeds to balance antipattern counts across a 34/33/33 project split. The selected seed was 767573.

The training set supported iterative prompt development and supplied few-shot examples, the validation set compared model/prompt combinations, and the test set was reserved for the final detector. The balancing objective and complete split construction are reported in Online Resource 1. The selected split had a score of 0.52.

Its test supports were 270 Implicit Columns, 105 ID Required, 36 Keyless Entry, 36 Fear of the Unknown, 39 31 Flavors, 21 Poor Man’s Search Engine, and 16 Rounding Errors occurrences. These denominators are reported with the detector metrics because class-specific estimates based on 16 cases carry visibly different sampling uncertainty from those based on 270. The data-dependent balancing is not a simple random partition, but the held-out projects remained unused during detector selection.

3.5 Model and Prompt Selection

We evaluated four contemporary models selected for architectural diversity, performance, repeatability, and structured-output support: OpenAI GPT-5.2, Z.ai GLM-5, Anthropic Claude Opus 4.5 with reasoning disabled, and OpenAI gpt-oss-120B (gpt 2025; GLM 2026; opu 2025; OpenAI 2025). Structured outputs were required so responses could be validated against a schema rather than parsed from free text (str 2024). The experiments were executed through OpenRouter as a unified provider gateway, with model fallbacks disabled (ope 2026).

The models represented closed reasoning, open reasoning, closed non-reasoning, and open-weight configurations. Requests were pinned to the requested model, and a local retry policy handled failed, empty, or schema-invalid responses. Provider-specific parameters, retry behaviour, and operational measurements are reported in Online Resource 1. The experiment therefore compares deployable configurations on one task rather than isolating model architecture as the sole variable.

The prompt design separated generated-schema files from hand-written query files. Schema prompts targeted database-design antipatterns and received additional constraint context needed for cases such as Keyless Entry. Query prompts targeted DML/DQL antipatterns in code interacting directly with jOOQ. Each input line was prefixed with its line number to reduce localisation errors. The final detector used Claude Opus 4.5 with reasoning disabled, temperature set to 0.0, and zero-shot prompts. This configuration was selected because the evaluated prompt-engineering variants did not consistently improve detection performance and increased cost or runtime.

Four strategies were compared on the validation set: zero-shot instructions, few-shot prompts, chain-of-thought prompting, and tree-of-thought prompting. Few-shot examples came primarily from training projects; synthetic examples covered variants absent from that set. Each combination analysed the same 20 validation projects and 823 relevant files. Accuracy, cost, runtime, and retry counts were recorded. The comparison was a single-run descriptive experiment: we did not test statistical significance or estimate run-to-run variance.

Every prompt provided a numbered source file, the applicable antipattern definitions, instructions to return an empty occurrence list for a negative file, and a schema restricting each occurrence’s label and inclusive bounds. Generated-schema prompts also received relevant key declarations for relational decisions. The response rationale was retained for qualitative analysis but did not enter the quantitative matching rule. Full prompt text and sample inputs are supplementary so the main manuscript can explain the protocol without relying on an inaccessible implementation detail.

Under zero-shot prompting, GPT-5.2, GLM-5, and Claude Opus 4.5 each achieved weighted F1 of 0.88 on validation, while gpt-oss-120B achieved 0.83. More elaborate prompts did not improve performance consistently and increased operational cost or runtime. We therefore selected the zero-shot Claude

Opus 4.5 configuration, which combined tied baseline accuracy, strong Key-less Entry precision, valid outputs, and the shortest runtime. Complete cost, runtime, retry, and prompt-strategy results are reported in Online Resource 1. This is a task-specific engineering choice, not a general ranking of the models or prompting methods.

3.6 Evaluation Protocol

We evaluate detection as a multi-label, multi-occurrence localisation task. Each prediction contains an antipattern type and inclusive line range. Given predicted line set S_p and ground-truth set S_g , line-span Intersection over Union (IoU) is

$$\text{IoU}(S_p, S_g) = \frac{|S_p \cap S_g|}{|S_p \cup S_g|}. \quad (1)$$

A prediction is a true positive when its antipattern type matches and $\text{IoU} \geq 0.5$. A wrong-label or below-threshold prediction is a false positive; an unmatched reference occurrence is a false negative. The threshold tolerates small span-boundary differences while requiring at least half of the combined source region to overlap. It is an operational choice for this study rather than a universal warning-localisation threshold.

Matching is one-to-one. When several predictions overlap a single same-label reference, Non-Maximum Suppression retains the prediction with the highest overlap as the true positive and counts the others as false positives. This prevents duplicate warnings from inflating recall. We calculate precision, recall, and F1 for each antipattern and report weighted totals using reference support. Absolute TP, FP, FN, and test support accompany the ratios so readers can reproduce the metrics and see class imbalance.

Specifically, $P = \text{TP}/(\text{TP} + \text{FP})$, $R = \text{TP}/(\text{TP} + \text{FN})$, and $F_1 = 2PR/(P + R)$. The support-weighted mean is $\sum_a N_a x_a / \sum_a N_a$, where $N_a = \text{TP}_a + \text{FN}_a$. We also report micro totals. Weighted averaging preserves per-class metrics while giving common classes greater influence; micro averaging pools decisions first. Neither replaces inspection of individual classes.

The final held-out run used Claude Opus 4.5, reasoning disabled, temperature 0.0, zero-shot schema/query prompts, structured output, and disabled provider fallbacks. It analysed 20 test projects containing 502 relevant files in 386 seconds at a cost of USD 14.34. The untouched human annotation is the primary reference. The corrected ground truth and a classification-only evaluation are secondary views: the former estimates the effect of known reference errors under an optimistic correction process, and the latter removes the requirement to localise multiple source spans. Neither quantifies model nondeterminism because the test call set was executed once.

3.7 Large-Scale Statistical Analysis

We applied the detector to all 602 projects in the final dataset, covering 17,450 relevant files. For each antipattern, we measured occurrence count, number of containing projects, occurrences per project, occurrences per containing project, and occurrences per 100 jOOQ statements. The final metric uses static jOOQ source references to jOOQ API methods that execute SQL as a statement proxy because loops, branches, reuse, and dynamic construction make executed statements in dynamic jOOQ code unreliable to count statically. It supports within-corpus comparison and does not estimate runtime or plain-SQL statement density. Full-run cost and elapsed time are reported in Online Resource 1.

The large-scale stage reused the detector without further prompt or model selection after observing test performance. Generated and non-generated files followed the same routing rules used during evaluation, and structured occurrences retained repository, path, antipattern, span, code fragment, and rationale fields for later aggregation. This ordering is important: adapting the detector to patterns seen in the 602-project output would entangle instrument development with the population being measured. Qualitative interpretation was performed after counts had been produced and did not relabel the corpus automatically.

Repository inclusion, file discovery, detector execution, and statistical aggregation are separate stages in the artefact. This separation permits a replication to substitute a newer model while retaining the corpus and metrics, or to rerun mining while retaining the detector configuration. It also exposes where a changed result originates. Reproducibility remains conditional on external repositories and model endpoints, but the procedural inputs are versioned rather than reconstructed from narrative alone.

To analyse co-occurrence, we computed Jaccard indices, conditional probabilities, and Spearman correlations at both project and file levels. To analyse API associations, we manually categorised the jOOQ API methods most frequently associated with query antipattern occurrences.

For sets A and B of projects or files containing two antipatterns, Jaccard similarity is $|A \cap B|/|A \cup B|$, while directional conditional probability is $P(B | A) = |A \cap B|/|A|$. Spearman's ρ uses ranked occurrence counts over the union of units containing either antipattern, including correction for tied ranks. Jaccard measures shared presence, conditional probability shows asymmetric containment, and Spearman measures monotonic association in intensity. None establishes causality, and high marginal prevalence can inflate the first two.

API association analysis was limited to Implicit Columns and Poor Man's Search Engine. Database-design cases mainly reflect generated schema output, while only 16 Fear of the Unknown detections (1.3%) appeared in query code. We iteratively matched occurrence snippets to concrete jOOQ or raw-SQL patterns until at least 98% received a label, retaining the residual as

Table 2 Held-out localisation performance against the original human ground truth. N is the number of reference test occurrences (TP+FN). The weighted mean uses per-antipattern reference support.

Antipattern	N	TP	FP	FN	P	R	F1
Implicit Columns	270	253	4	17	0.98	0.94	0.96
ID Required	105	89	13	16	0.87	0.85	0.86
Keyless Entry	36	35	30	1	0.54	0.97	0.69
Fear of the Unknown	36	19	24	17	0.44	0.53	0.48
31 Flavors	39	38	1	1	0.97	0.97	0.97
Poor Man’s Search Engine	21	13	4	8	0.76	0.62	0.68
Rounding Errors	16	13	0	3	1.00	0.81	0.90
All occurrences (micro)	523	460	76	63	0.86	0.88	0.87
Weighted mean	–	–	–	–	0.88	0.88	0.88

other/uncategorised. These labels describe where detections manifest, not whether an API method causes an antipattern.

All aggregation uses static repository snapshots and gives each repository or file one row at the corresponding analysis level. Project-level presence is binary for Jaccard and conditional probability, whereas Spearman uses the detected occurrence count. File-level analysis follows the same distinction. We report selected patterns in the main text and preserve complete matrices in Online Resource 1. The analysis did not adjust occurrence-count relationships for repository size, and no separate manual audit validated detector outputs from the 602-project population. The matrices are therefore exploratory co-detection summaries, not inferential or size-controlled associations.

4 Results

4.1 RQ1: How accurately can LLMs localise SQL antipattern occurrences in jOOQ-based Java code?

We first evaluate whether the selected LLM detector can support the large-scale study as a research instrument. Table 2 reports the held-out localisation results against the original human ground truth. Absolute counts and test supports are primary: they expose the varying amounts of evidence available for each antipattern and allow every ratio to be reconstructed.

The detector achieved weighted precision, recall, and F1 of 0.88. Across all classes it matched 460 of 523 reference occurrences and produced 76 unmatched predictions. The micro F1 of 0.87 is close to the support-weighted value, so the main result is not an artefact of reporting only one averaging convention. Nevertheless, aggregate values conceal a wide class range: F1 spans from 0.48 for Fear of the Unknown to 0.97 for 31 Flavors.

The hardest antipatterns were Keyless Entry and Fear of the Unknown. Both require contextual judgement about schema relationships, business concepts, or the meaning of default and nullable values. For example, Keyless

Entry requires deciding whether an apparent identifier column should refer to another table and whether an appropriate key exists. Fear of the Unknown requires distinguishing genuinely nullable values from special defaults or nullable declarations that are semantically non-nullable. These cases expose the main limitation of file-local prompting: some decisions depend on project conventions and domain semantics that may not be present in the analysed file.

The two hard classes failed differently. Keyless Entry recall was 0.97, but its 30 false positives reduced precision to 0.54. The detector frequently inferred a relationship from column naming or domain similarity that the annotation rules did not consider sufficiently supported. Fear of the Unknown had both low precision (0.44) and recall (0.53), reflecting ambiguity in whether a default has a legitimate business meaning and whether a nullable declaration can in practice receive null. These are not simply boundary-localisation errors; many require evidence outside the presented file. Adding more verbose instructions is unlikely to recover absent project semantics.

Manual error inspection identified recurring sources of disagreement. Audit columns and technical identifiers were sometimes interpreted as domain references; columns with names suggesting a relationship did not always have a suitable target; and generated defaults could represent framework conventions rather than missing-value sentinels. For query antipatterns, the model sometimes inferred wildcard intent from a method name without seeing caller-provided values, or omitted a case when string construction was separated from the predicate. These patterns explain why file-local semantic flexibility improves coverage but cannot substitute for explicit project retrieval.

The strongest performance appeared for antipatterns with more local syntactic or API-level signals, such as Implicit Columns and 31 Flavors. This pattern supports the use of LLMs as a flexible detector for source-level antipatterns, but it also suggests that future improvements should focus on retrieval or static pre-analysis for context-dependent cases rather than simply adding more prompt instructions.

Implicit Columns combined precision 0.98 and recall 0.94 across 270 test occurrences, indicating that the model usually recognised both explicit and API-mediated blind projections. 31 Flavors achieved F1 0.97, although its support was only 39. Rounding Errors produced no false positives and three false negatives among 16 occurrences. The latter perfect precision should therefore be read with its denominator: a few additional errors would change it substantially. Poor Man’s Search Engine showed the opposite practical concern, missing eight of 21 reference cases, often where wildcard intent was indirect.

Ground-truth sensitivity analysis Reviewing detector disagreements corrected known omissions or mistakes in the reference data. Against that revised variant, weighted precision increased from 0.88 to 0.96, recall from 0.88 to 0.91, and F1 from 0.88 to 0.93. The largest changes occurred for context-sensitive schema classes: Keyless Entry F1 rose from 0.69 to 0.86 and Fear of the Unknown from 0.48 to 0.82.

Reference uncertainty therefore materially affects class-level conclusions. The revised variant remains biased because detector-disagreed cases received disproportionate review and jointly missed occurrences remain unknown. We retain 0.88 as the primary result and use 0.93 only to show the direction and possible magnitude of known annotation error.

Classification mode achieved a similar weighted F1-score: 0.88 against the original ground truth and 0.94 against the corrected ground truth. It is nevertheless less useful as a static-analysis output because it cannot distinguish multiple occurrences in the same file or identify the exact code range that should be inspected. We therefore treat classification as a secondary robustness check and use the localisation mode as the primary instrument for the large-scale study.

This comparison suggests that requiring line spans did not reduce the aggregate weighted F1 relative to classification in this experiment. It should not be interpreted as equivalence between tasks: classification collapses multiple same-type occurrences and cannot penalise imprecise warning locations. The localisation counts remain the appropriate evidence for a detector whose outputs are later counted as individual occurrences.

The validation-to-test transition also provides a limited check against configuration selection bias. The chosen model achieved validation weighted F1 of 0.88 under zero-shot prompting and the final localised test F1 was likewise 0.88, despite the test set containing different projects and 502 rather than 823 files. Similar aggregates do not demonstrate distributional equivalence, but the absence of a large collapse supports the decision to use the held-out detector for corpus analysis. Class-level denominators remain the more informative evidence.

Because all 523 original test occurrences contribute to the weighted estimate, its apparent two-decimal precision should not be mistaken for exact population performance. Scores for low-support classes remain especially fragile, and all model calls were single observations. We use two decimals for consistent reporting and base downstream confidence on the combination of counts, error types, and class support rather than on small differences in rounded F1.

Answer to RQ1 On 523 held-out reference occurrences, the selected zero-shot Claude Opus 4.5 detector achieved weighted localisation F1 of 0.88 against the original human ground truth. Performance was strongest for locally expressed antipatterns and weakest for definitions requiring project or domain context. The result supports using the detector as a fallible large-scale research instrument, provided that prevalence findings are phrased as detected occurrences and class-specific error differences remain explicit.

4.2 RQ2: How prevalent are SQL antipatterns in open-source jOOQ projects?

The large-scale analysis flagged 15,931 occurrences across the 602 projects. Table 3 reports both corpus-wide prevalence and concentration within affected

Table 3 Prevalence of SQL antipatterns in 602 analysed jOOQ projects.

Antipattern	Occurrences	Projects	Per project	Per containing project	Per 100 jOOQ statements
Implicit Columns	7,289	535	12.11	13.62	21.94
ID Required	3,597	523	5.98	6.88	10.83
Keyless Entry	2,153	200	3.58	10.77	6.48
Fear of the Unknown	1,256	205	2.09	6.13	3.78
31 Flavors	390	87	0.65	4.48	1.17
Poor Man’s Search Engine	583	114	0.97	5.11	1.75
Rounding Errors	663	86	1.10	7.71	2.00
Total	15,931	601	26.46	26.51	47.96

projects. We use “flagged” or “detected” deliberately: the counts are outputs of an instrument whose held-out performance differs by antipattern, not a manually verified census of every occurrence.

Implicit Columns and ID Required dominate the detected occurrences. Implicit Columns appears in 535 projects, or approximately 89% of the dataset, while ID Required appears in 523 projects, or approximately 87%. The remaining antipatterns appear in fewer projects but remain concentrated within affected projects. For example, Keyless Entry appears in 200 projects but averages 10.77 occurrences within those projects.

Only one project contained no detected occurrence, whereas 601 contained at least one of the seven types. The total average of 26.46 detections per project is not representative of a typical project because repository sizes are strongly skewed. For this reason, Table 3 also shows the number of containing projects and the mean within those projects. The two dominant classes account for 10,886 detections, or 68.3% of the total. Their near-ubiquity means that project-level presence alone provides little discrimination; occurrence counts and source manifestations are more informative.

The lower-prevalence classes remain visible as detector outputs, but the evidential strength differs. Rounding Errors and 31 Flavors each appear in approximately one seventh of projects, while Poor Man’s Search Engine appears in 114 projects. Keyless Entry and Fear of the Unknown appear in 200 and 205 projects, respectively, but their held-out precision was only 0.54 and 0.44 against the original reference data. Their corpus counts should therefore be treated as low-confidence flags for follow-up, not estimates comparable in reliability to Implicit Columns.

The statement-normalised column preserves the ordering of the two dominant classes but does not measure runtime frequency. Implicit Columns has 21.94 detected occurrences per 100 static statement proxies and ID Required has 10.83. Schema antipatterns and query-execution proxies also have different conceptual denominators: a generated column declaration can represent a persistent schema decision, whereas an executing API reference approximates a query site. The rates therefore provide only a common within-study scale; they do not imply that nearly half of executed SQL statements are antipattern occurrences.

Table 4 Selected unadjusted co-detection patterns. Values describe detector outputs and do not control for project size.

Pattern	Jaccard	Spearman	Interpretation
Implicit Columns and ID Required	0.81	0.43	High-prevalence outputs whose counts may also rise with repository size.
Implicit Columns and Poor Man’s Search Engine	–	positive at file level	Query detections appear together in some database-access files; the complete value is reported in Online Resource 1.

Detector error introduces unequal uncertainty. Implicit Columns, 31 Flavors, and Rounding Errors had high held-out F1, increasing confidence in their relative counts under the study definitions. Keyless Entry and Fear of the Unknown had substantially lower original-ground-truth precision, so their large-scale counts may contain many false positives. No independent sample of population outputs was manually validated. We do not apply a global accuracy correction: class-level errors are not necessarily random, the annotated projects differ from the complete corpus, and corrected-ground-truth evaluation is optimistic. Reporting raw detected counts with class-specific validation evidence is more transparent than producing pseudo-adjusted prevalence estimates.

Answer to RQ2 The detector flagged at least one occurrence in 601 of 602 analysed open-source jOOQ projects and 15,931 occurrences overall. Implicit Columns and ID Required dominate the outputs, appearing in 535 (89%) and 523 (87%) projects. These are detector-output measurements rather than a manually validated population census; Keyless Entry and Fear of the Unknown are especially low-confidence population estimates.

4.3 RQ3: Which SQL antipatterns co-occur across projects and files?

We summarise selected project- and file-level co-detection patterns. The full Jaccard, conditional-probability, and Spearman matrices are provided in Online Resource 1 so that the selected values can be checked against all pairwise results. These statistics are unadjusted for project size.

The largest selected project-level values are for Implicit Columns and ID Required, with a Jaccard index of 0.81 and Spearman correlation of 0.43 (Table 4). Both outputs are individually prevalent, so substantial overlap is expected even without a shared mechanism.

The distinction between presence and intensity matters here. Because both antipatterns occur in nearly nine out of ten projects, their Jaccard similarity would be high even if the mechanisms producing them were unrelated. Their counts also have more opportunities to rise in larger repositories. Because the analysis did not use partial correlation, regression, stratification, or another

size adjustment, the Spearman value cannot distinguish a relationship between the antipatterns from shared exposure to project size.

Keyless Entry and Fear of the Unknown also co-detected in the raw matrices, but both labels had weak held-out precision and require contextual interpretation. We therefore do not infer a corpus-wide “constraint neglect” construct from that pair. Its unadjusted values remain in Online Resource 1 for transparency and hypothesis generation.

At the file level, design and query antipatterns tend to separate because they are often located in generated schema classes and database access classes, respectively. This separation is expected from the structure of jOOQ projects and should not be interpreted as evidence that schema and query problems are unrelated at the project level. Query antipatterns are more likely to co-occur within the same files, particularly where fluent jOOQ calls concentrate data retrieval logic.

Directional conditional probabilities are similarly dominated by the baseline prevalence of common outputs. For example, a high probability of Implicit Columns given a rarer query antipattern provides little evidence beyond Implicit Columns already appearing in most projects. We therefore retain these values as descriptive summaries rather than predictors.

Project- and file-level results answer different questions. Project-level overlap indicates that practices coexist somewhere in the same repository; it may reflect teams, schema conventions, size, or shared infrastructure without the occurrences being adjacent. File-level overlap locates practices in the same generated class or access component, but structural separation between generated schema and hand-written query code mechanically lowers cross-family similarity. Comparing both levels prevents a low file-level score from being mistaken for absence of a project-wide relationship.

Answer to RQ3 The unadjusted detector outputs show their largest selected project-level overlap for Implicit Columns with ID Required (Jaccard 0.81; Spearman $\rho = 0.43$). High marginal prevalence and uncontrolled repository size limit its interpretation. File-level values also reflect the structural separation of generated-schema and hand-written query files. RQ3 therefore yields exploratory co-detection patterns, not size-independent or causal relationships.

4.4 RQ4: Which jOOQ API methods are most frequently associated with query antipattern occurrences?

We connect query-antipattern occurrences to concrete jOOQ API usage patterns. Table 5 reports the two antipatterns with sufficient query-code support. For Implicit Columns, calls that fetch a complete generated record account for most detections. For Poor Man’s Search Engine, the dominant methods construct wildcard predicates behind a fluent API.

These associations do not prove that the API causes the antipatterns. They do, however, identify concrete API surfaces where developer guidance,

Table 5 Most frequent jOOQ API associations for selected query antipatterns.

Antipattern	Associated method or pattern	Occurrences	Share
Implicit Columns	DSL/DSLContext.selectFrom(TABLE)	3,417	46.9%
Implicit Columns	DSL/DSLContext.select().from(TABLE)	1,810	24.8%
Implicit Columns	TABLE.fields()	665	9.1%
Implicit Columns	DSL.asterisk()	280	3.8%
Implicit Columns	Generated DAO methods	163	2.2%
Implicit Columns	DSL/DSLContext.fetch(TABLE, ...)	151	2.1%
Implicit Columns	Plain SQL wildcards	137	1.9%
Implicit Columns	DSL/DSLContext.fetchOne(TABLE, ...)	130	1.8%
Implicit Columns	DSL/DSLContext.select(TABLE).from(TABLE)	107	1.5%
Implicit Columns	Other/uncategorised	429	5.9%
Poor Man’s Search Engine	Field.like	273	46.8%
Poor Man’s Search Engine	Field.containsIgnoreCase	96	16.5%
Poor Man’s Search Engine	Field.likeIgnoreCase	87	14.9%
Poor Man’s Search Engine	Field.contains	61	10.5%
Poor Man’s Search Engine	Plain SQL LIKE	17	2.9%
Poor Man’s Search Engine	Field.likeRegex	12	2.1%
Poor Man’s Search Engine	Other/uncategorised	37	6.3%

documentation warnings, examples, or future static-analysis rules could be targeted.

The two most frequent Implicit Columns forms, `selectFrom(TABLE)` and `select().from(TABLE)`, account for 5,227 occurrences (71.7%). Adding `TABLE.fields()` raises the covered share to 80.8%. The long tail contains explicit asterisks, generated DAO retrieval, convenience fetch methods, raw SQL, and less common fluent combinations. This concentration matters for tool design: a small deterministic rule set could capture much of the most frequent class, while an LLM or broader data-flow analysis could be reserved for uncommon or context-dependent forms.

For Poor Man’s Search Engine, `Field.like` accounts for 273 occurrences (46.8%). `containsIgnoreCase`, `likeIgnoreCase`, and `contains` together contribute another 244 (41.9%). These higher-level names can make wildcard semantics less visually explicit than raw `LIKE`. The association does not make the calls inherently wrong: prefix matching and deliberately small domains remain legitimate, and the operational definition depends on available intent evidence. Documentation examples can nevertheless make the distinction between pattern matching and database full-text search more visible.

The iterative categorisation left 5.9% of Implicit Columns and 6.3% of Poor Man’s Search Engine detections in the other/uncategorised group. Retaining those categories exposes the coverage ceiling rather than forcing unusual ex-

pressions into dominant labels. Because the categories were derived from the same occurrence corpus, their shares are descriptive and should be validated before being used as a general taxonomy of jOOQ usage.

Answer to RQ4 Implicit Columns is most often associated with `selectFrom(TABLE)` and `select().from(TABLE)`, which together account for 71.7% of its detections. Poor Man’s Search Engine is dominated by `Field.like`, `containsIgnoreCase`, `likeIgnoreCase`, and `contains`, together accounting for 88.7%. These concentrations identify practical rule and documentation targets without establishing that the APIs cause the antipatterns.

5 Discussion

5.1 What the Findings Add

The operationalised patterns recur across the analysed open-source jOOQ projects. Implicit Columns and ID Required are flagged in nearly nine out of ten repositories. Implicit Columns is the stronger population finding because its held-out precision and recall were both high. Many occurrences arise through jOOQ idioms that fetch generated records or all table fields, beyond literal `SELECT *` strings. This finding supports source-level analysis that understands API semantics alongside extracted SQL strings.

Prevalence under an operational definition does not imply uniform harm. A broad projection may be harmless for a small stable table or deliberate when a complete generated record is required. A generic `id` key may be consistent with a project’s modelling conventions even when a natural key exists. Antipattern catalogues are diagnostic heuristics that identify decisions worth reviewing. The large-scale counts therefore characterise recurring design and access patterns and help prioritise further investigation.

Localisation produces inspectable warning regions. Evaluating line spans with IoU distinguishes a file-level recognition from an actionable source region, which is especially important when one file contains several occurrences of the same type. The IoU gate tolerates small differences in whether a fluent call’s receiver or closing line is included while penalising spans that cover an entire class. One-to-one matching also penalises repeated warnings for the same site. Developer studies remain necessary because overlap with an annotation is only a proxy for the usefulness of returned spans and explanations.

5.2 Relationship to Existing Detection Approaches

SQLInspect extracts SQL from Java code, so the query extractor’s reported performance limits the antipattern detector built on top of it (Nagy and Cleve 2017; Meurice et al. 2016; Muse et al. 2020). Our LLM-based approach inspects Java source and generated jOOQ classes directly. It can therefore detect blind

projections expressed as jOOQ convenience calls and schema-level Fear of the Unknown occurrences outside the DML/DQL scope of SQLInspect. The tools target different representations, antipattern subsets, and evaluation designs, which precludes a direct head-to-head comparison.

Dedicated API or AST rules would be preferable for stable high-volume forms. The API-association results show that two or three patterns cover most Implicit Columns detections, and those patterns do not require probabilistic interpretation. Deterministic checks are cheaper, easier to reproduce, and easier for developers to trust. Their limitation is coverage: aliases, intermediate query objects, raw SQL mixtures, generated-schema semantics, and context-dependent relationships require additional data flow or project modelling. A practical successor should therefore be hybrid. Rules can handle high-confidence syntactic cases, while an LLM receives only cases needing semantic context and can be evaluated under the same localisation contract.

Prior studies already demonstrate classification of code smells and compare prompting or fine-tuning (Sousa et al. 2025; Sadik and Govind 2025; Alomari et al. 2026). Our evidence concerns a task-specific engineering choice: zero-shot was selected because more elaborate strategies did not consistently improve this task and were operationally expensive. The distinct contribution is the use of a held-out, occurrence-localised detector as an explicitly validated instrument for a repository study. Su and McMillan’s finding that coding performance does not imply conventional static-analysis capability reinforces the need for this task-specific validation (Su and McMillan 2026).

Muse et al. report Implicit Columns and Fear of the Unknown as percentages of SQL statements (Muse et al. 2020). Our density metric instead counts occurrences per 100 static jOOQ statement proxies, and a single generated or fluent source construct can correspond to runtime behaviour that is hard to count statically. The larger observed rates indicate that jOOQ exposes additional source-level manifestations. They cannot establish that jOOQ projects are categorically more prone to SQL antipatterns than plain-SQL projects.

5.3 Implications for Practice and Research

Two findings guide practice. First, stable syntactic cases support deterministic rules. The dominant Implicit Columns associations point to convenience methods that make broad projections easy and idiomatic. Poor Man’s Search Engine associations show that methods such as `contains` and `containsIgnoreCase` can hide wildcard-based matching behind a higher-level API. For jOOQ users, documentation and examples can distinguish appropriate use from cases requiring explicit projections or full-text search. Warnings can state the selected table or search pattern and suggest the corresponding alternative.

Second, contextual cases require project evidence. Keyless Entry and Fear of the Unknown may require migrations, neighbouring tables, naming conventions, or developer confirmation. Schema warnings should identify the upstream constraint and suppress views or intentional defaults because generated

classes are normally edited through their source schemas. Class-specific confidence thresholds, suppression, and baseline support can prevent high-volume existing occurrences from overwhelming developers. These workflow features remain unevaluated, so the results establish detector feasibility and research-instrument validity without establishing readiness for continuous integration.

Repository mining also requires validation at the same granularity used for counting. A file classifier cannot justify occurrence totals, and aggregate F1 can obscure a class with many false positives. Publishing test supports, absolute counts, prompt definitions, correction provenance, and full co-detection matrices lets readers assess the downstream findings. A replication would benefit most from independent annotation of the 602-project outputs, repeated LLM runs, and size-controlled analysis because this study omitted those three checks.

A staged evidence standard follows. Labels should be defined in the analysed source representation, prompt development should remain separate from project-level held-out testing, and reports should include absolute occurrence-matching results. Mined claims should then follow the weakest relevant class validation. This standard permits flexible analysis while exposing measurement error. The observed context-dependent errors further motivate retrieval over project context, explicit schema relationship modelling, or hybrid static-analysis prefilters. A practical tool would also require incremental analysis and developer-facing explanations tested in realistic maintenance tasks.

The results are a snapshot because model identifiers, provider routing, repository contents, and jOOQ idioms change. The study preserves parameters, prompts, selection dates, and artefacts so the procedure can be repeated. A future replication may reasonably obtain different absolute scores or corpus membership. The durable contribution is the operationalised task and linked validation/mining design; the individual model ranking is specific to this snapshot.

6 Threats to Validity

Construct validity The principal construct risk concerns whether the labels represent SQL antipatterns consistently in jOOQ source. Several definitions adapt plain-SQL concepts to generated classes and fluent APIs, which may differ from other studies. We mitigated ambiguity with explicit inclusion/exclusion rules, decision trees, re-review after guideline changes, and publication of the codebook.

A 128-file washout re-annotation obtained exact-span $\kappa = 0.834$, supporting temporal consistency. Because one annotator produced both passes, this result does not establish expert consensus. The corrected-ground-truth sensitivity analysis shows that reference errors can materially change class scores. It is optimistic because detector disagreements triggered review. Prevalence therefore applies to these operational definitions, and cases missed by both human and model remain unmeasured.

The IoU threshold introduces a second construct choice. A threshold of 0.5 tolerates small boundary differences, while exact matching would answer a stricter question. We mitigate duplicate and overly broad matches with same-label one-to-one NMS and publish the formula and spans. No alternative-threshold sensitivity run was performed, so the reported scores remain conditional on 0.5. Classification results show that removing localisation does not change aggregate weighted F1. They leave the threshold itself unvalidated.

Internal validity Prompt and model selection may overfit the validation set, and single LLM runs can vary. We separated projects across training, validation, and test sets, used whole-project assignment, fixed all sampling/splitting seeds, reserved the test set for the final configuration, constrained outputs by schema, and disabled provider fallbacks. Few-shot examples came mainly from training data, although synthetic edge cases and repeated prompt iteration may still encode researcher expectations. We performed no significance tests and no repeated-run variance analysis; model/prompt comparisons are descriptive. The held-out result reduces direct test overfitting. General claims that zero-shot prompting or Claude Opus 4.5 is superior outside this task remain unsupported.

External validity The corpus contains open-source GitHub Java projects that declare jOOQ dependencies and commit generated classes. We mitigate selection opacity by publishing exact search terms, the dated collection point, funnel counts, duplicate categories, and size-stratified sampling. Specific manifest queries reduce but cannot eliminate losses from GitHub’s 1,000-result cap. Maven/Gradle selection excludes other build systems, and committed generated classes may distinguish the corpus from enterprise practice. No sensitivity corpus using another search source was available. Claims therefore apply to the analysed open-source population. They exclude closed-source systems, build-time-only generation, other languages, and query builders such as LINQ and SQLAlchemy.

Conclusion validity Absolute detector counts inherit class-specific precision and recall, and co-detection measures are affected by repository size and marginal prevalence. We report TP/FP/FN and test supports, present three complementary summaries at project and file levels, retain uncategorised API cases, and distinguish observation from interpretation. We do not accuracy-adjust the corpus counts because errors need not be random or stable across populations. We also performed no separate manual validation of population outputs, so the held-out class estimates may not transfer unchanged to the 602-project corpus. Keyless Entry and Fear of the Unknown counts are consequently treated as low-confidence flags rather than equivalent population estimates.

The co-detection analysis did not control for project size. Larger projects create more opportunities for every antipattern, so count correlations can arise

from shared exposure even without a direct relationship. Reporting both presence and count summaries and noting marginal prevalence makes this risk visible but does not remove it. The RQ3 findings are therefore hypothesis-generating unadjusted patterns; a size-stratified analysis, partial correlation, or count model with an exposure term would be required for a size-independent claim.

Occurrences per 100 jOOQ statements use static SQL-executing API references. Loops and control flow can change actual runtime frequency. Accordingly, the claims concern detected prevalence and descriptive API associations. They exclude causal effects, statistically significant clusters, and direct density comparisons with plain SQL.

The study did not compute confidence intervals or hypothesis tests for model, prompt, prevalence, or co-occurrence differences. We report this omission explicitly because the archived outputs do not support the assumptions needed for a post hoc repair. Absolute denominators and complete matrices allow future project-level bootstrap or repeated-run analyses. Until then, small metric differences and weak correlations are treated as descriptive, and substantive interpretation focuses on large, repeated patterns with clear operational meaning.

API categories were developed from observed detections without a preregistered taxonomy. Achieving at least 98% coverage and retaining an uncategorised group makes the procedure inspectable, although the categories may reflect this corpus. The result supports prioritising prominent methods for further study. It does not provide a complete universal classification of jOOQ APIs.

Reliability and reproducibility The experiments used commercial and gateway-hosted endpoints. Provider updates, backend routing, quantisation, caching, and transient failures can affect replications. We recorded model identifiers and parameters, disabled automatic fallbacks, used structured outputs, separated invalid-response retries, retained prompts and code, and selected point-in-time snapshots where possible. The full 602-project run also records cost and elapsed time. These mitigations improve auditability but cannot recreate provider internals or guarantee deterministic output. Replications should report both accuracy and operational behaviour and should not treat exact numerical reproduction as expected from a conventional static analyser.

7 Conclusion

This paper validated LLM-based localisation of SQL antipatterns in jOOQ database access code and used the resulting instrument for repository mining within the analysed corpus. We constructed a manually annotated dataset of 1,562 antipattern occurrences, evaluated detection as a line-span localisation task, and applied the selected detector to 602 open-source jOOQ projects. Against the original held-out ground truth, the detector achieved weighted F1

of 0.88 across 523 reference occurrences. The large-scale run flagged 15,931 occurrences, with Implicit Columns and ID Required appearing in nearly 90% of projects.

The findings show that SQL antipatterns in jOOQ projects arise through both API-level constructs and explicit SQL strings. They also show that LLM-based localisation can complement traditional extraction-based static analysis for dynamic database access libraries. The strongest future opportunities are hybrid detectors that combine LLM source understanding with stronger project-context retrieval, schema modelling, and reproducibility controls.

The scope reflects a single annotator, class-dependent detector error, no separate population-output audit, unadjusted project-size exposure, an open-source corpus that requires committed generated classes, and a single execution of each LLM experiment. Within this scope, the study provides a reproducible localisation protocol and empirical baseline for future independent annotation, repeated-run evaluation, size-controlled mining, hybrid rule/LLM detectors, and developer-centred assessment of warning usefulness.

Supplementary Material

Online Resource 1, *Supplementary Material for “Validating LLM-Based Localisation for Mining SQL Antipatterns in jOOQ Repositories”*, provides the split-optimisation details, provider configurations and operational measurements, full prompt templates, annotation decision trees, per-antipattern confusion counts, complete Jaccard/Spearman/conditional-probability matrices, GitHub search terms, and omitted-project list.

Statements and Declarations

Prior dissemination and relationship to the master’s thesis This article is based on Kristo Isberg’s master’s thesis, *Detecting SQL Antipatterns in jOOQ Database Access Code Using Large Language Models* (Isberg 2026). The repository corpus, annotated dataset, detector implementation, model and prompt experiments, and large-scale measurements originate in that work and are not presented as newly collected or as an independent replication. This article selects and synthesises that evidence around an occurrence-localised measurement protocol, updates its positioning against related research, and carries class-specific instrument limitations into detector-output repository findings.

Corresponding author [CORRESPONDING AUTHOR CONFIRMATION REQUIRED: Kristo Isberg is assumed in the drafting notes, but the corresponding author must be approved and explicitly marked before submission.]

Author contributions [AUTHOR CONFIRMATION REQUIRED: specify Kristo Isberg’s and Erki Eessaar’s contributions using free-text or CRediT roles. The thesis identifies Kristo Isberg as the student author and Erki Eessaar as supervisor, but manuscript contribution roles must be approved by both authors.] All authors must read and approve the final manuscript.

Competing interests The authors declare that they have no competing interests.

Funding No funding was received for conducting this study.

Data availability The data supporting the findings, including the annotated dataset, experimental outputs, analysis results, prompts, and source-material references, are publicly available at <https://github.com/kristoisberg/masters-thesis>. The corpus was collected on 10 January 2026. [ARCHIVAL IDENTIFIER CONFIRMATION REQUIRED: deposit and cite a DOI-backed snapshot and record the exact source repository commit before submission.]

Code and materials availability The SQL-antipattern detector and its source code are publicly available at <https://github.com/kristoisberg/jooq-antipattern-detector>. Search terms, annotation decision trees, prompt templates, confusion matrices, co-detection matrices, and omitted-project details are provided in Online Resource 1 and the artefact repository. [DETECTOR COMMIT CONFIRMATION REQUIRED: record the exact analysed revision before submission.]

Ethics approval Not applicable. The study analysed publicly available software repositories and did not involve human participants or animals.

Consent to participate Not applicable.

Use of artificial intelligence Generative artificial intelligence tools were used as supportive aids during the research and manuscript preparation process. Google Gemini, Google NotebookLM, and OpenAI Codex were used to support drafting, restructuring, wording improvements, code development, and artefact preparation. The authors critically reviewed, fact-checked, and edited all AI-assisted content and take full responsibility for the accuracy, originality, and integrity of the manuscript.

References

- (2022) Rest api endpoints for search. URL <https://docs.github.com/en/rest/search/search?apiVersion=2022-11-28>
- (2024) Introducing structured outputs in the api. URL <https://openai.com/index/introducing-structured-outputs-in-the-api/>
- (2025) Sql: Select *. URL <https://www.jooq.org/doc/latest/manual/reference/dont-do-this/dont-do-this-sql-select-all/>

- (2025) System card: Claude opus 4.5. URL <https://www-cdn.anthropic.com/bf10f64990cfda0ba858290be7b8cc6317685f47.pdf>
- (2025) Update to gpt-5 system card: Gpt-5.2. URL <https://deploymentsafety.openai.com/gpt-5-2/introduction>
- (2026) Openrouter. URL <https://openrouter.ai/>
- Alomari N, Redah M, Ashraf A, Alshayeb M (2026) Smelly-shot is all you need: an empirical study to compare in-context learning verses fine tuning for code smell detection. *Empirical Software Engineering* 31(185), DOI 10.1007/s10664-026-10915-2, URL <https://doi.org/10.1007/s10664-026-10915-2>
- Blaauwendraad R (2025) Evaluating large language models for code smell detection. Bachelor's thesis, Radboud University, URL https://www.cs.ru.nl/bachelors-theses/2025/Robert_Blaauwendraad___1084960___Evaluating_Large_Language_Models_for_Code_Smell_Detection.pdf
- Dintyala P, Narechania A, Arulraj J (2020) Sqlcheck: Automated detection and diagnosis of SQL anti-patterns. In: Maier D, Pottinger R, Doan A, Tan W, Alawini A, Ngo HQ (eds) *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference 2020*, online conference [Portland, OR, USA], June 14-19, 2020, ACM, pp 2331-2345, DOI 10.1145/3318464.3389754
- Fontana FA, Mäntylä MV, Zanoni M, Marino A (2016) Comparing and experimenting machine learning techniques for code smell detection. *Empir Softw Eng* 21(3):1143-1191, DOI 10.1007/S10664-015-9378-4
- GLM (2026) GLM-5: from vibe coding to agentic engineering. *CoRR abs/2602.15763*, DOI 10.48550/ARXIV.2602.15763, 2602.15763
- Isberg K (2026) Detecting SQL antipatterns in jOOQ database access code using large language models. Master's thesis, Tallinn University of Technology, URL <https://github.com/kristoisberg/masters-thesis>
- Karwin B (2023) SQL antipatterns, volume 1. Pragmatic Programmers, Raleigh, NC
- Lyu Y, Alotaibi A, Halfond WGJ (2019) Quantifying the performance impact of SQL antipatterns on mobile applications. In: 2019 IEEE International Conference on Software Maintenance and Evolution, ICSME 2019, Cleveland, OH, USA, September 29 - October 4, 2019, IEEE, pp 53-64, DOI 10.1109/ICSME.2019.00015
- Marinescu R (2004) Detection strategies: Metrics-based rules for detecting design flaws. In: 20th International Conference on Software Maintenance (ICSM 2004), 11-17 September 2004, Chicago, IL, USA, IEEE Computer Society, pp 350-359, DOI 10.1109/ICSM.2004.1357820
- Meurice L, Nagy C, Cleve A (2016) Static analysis of dynamic database usage in java systems. In: Nurcan S, Soffer P, Bajec M, Eder J (eds) *Advanced Information Systems Engineering - 28th International Conference, CAiSE 2016*, Ljubljana, Slovenia, June 13-17, 2016. *Proceedings, Springer, Lecture Notes in Computer Science*, vol 9694, pp 491-506, DOI 10.1007/978-3-319-39696-5_30
- Moha N, Guéhéneuc Y, Duchien L, Meur AL (2010) DECOR: A method for the specification and detection of code and design smells. *IEEE Trans Software Eng* 36(1):20-36, DOI 10.1109/TSE.2009.50
- Muse BA, Rahman MM, Nagy C, Cleve A, Khomh F, Antoniol G (2020) On the prevalence, impact, and evolution of SQL code smells in data-intensive systems. In: Kim S, Gousios G, Nadi S, Hejderup J (eds) *MSR '20: 17th International Conference on Mining Software Repositories*, Seoul, Republic of Korea, 29-30 June, 2020, ACM, pp 327-338, DOI 10.1145/3379597.3387467
- Nagy C, Cleve A (2017) A static code smell detector for SQL queries embedded in java code. In: 17th IEEE International Working Conference on Source Code Analysis and Manipulation, SCAM 2017, Shanghai, China, September 17-18, 2017, IEEE Computer Society, pp 147-152, DOI 10.1109/SCAM.2017.19
- Nagy C, Cleve A (2018) Sqlinspect: a static analyzer to inspect database usage in java applications. In: Chaudron M, Crnkovic I, Chechik M, Harman M (eds) *Proceedings of the 40th International Conference on Software Engineering: Companion Proceedings, ICSE 2018*, Gothenburg, Sweden, May 27 - June 03, 2018, ACM, pp 93-96, DOI 10.1145/3183440.3183496

- OpenAI (2025) gpt-oss-120b & gpt-oss-20b model card. CoRR abs/2508.10925, DOI 10.48550/ARXIV.2508.10925, 2508.10925
- Sadik AR, Govind S (2025) Benchmarking LLM for code smells detection: Openai GPT-4.0 vs deepseek-v3. CoRR abs/2504.16027, DOI 10.48550/ARXIV.2504.16027, 2504.16027
- Sharma T, Fragkoulis M, Rizou S, Bruntink M, Spinellis D (2018) Smelly relations: measuring and understanding database schema quality. In: Paulisch F, Bosch J (eds) Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice, ICSE (SEIP) 2018, Gothenburg, Sweden, May 27 - June 03, 2018, ACM, pp 55–64, DOI 10.1145/3183519.3183529
- Sousa V, Baptista C, Alves AL, Figueirêdo H (2025) Effectiveness of small and large language models for pl/sql bad smell detection. In: Anais do XL Simpósio Brasileiro de Bancos de Dados, SBC, Porto Alegre, RS, Brasil, pp 399–412, DOI 10.5753/sbbd.2025.247256
- Su CY, McMillan C (2026) Do code LLMs do static analysis? Empirical Software Engineering 31(116), DOI 10.1007/s10664-026-10853-z, URL <https://doi.org/10.1007/s10664-026-10853-z>
- Tamberg K, Bahsi H (2024) Suurte keelemudelite kasutamine turvanõrkuste tuvastamiseks lähtekoodis: põhjalik võrdlusuuring. Master's thesis, URL <https://digikogu.taltech.ee/et/item/a0cfeda0-dc43-4408-8116-2fb657ed9a44>